

CityVision

Segmentation sémantique de scènes urbaines
et mise en production sur le Cloud

Darya Filatova

Projet 8 — Ingénieur IA

27 juillet 2026

Cityscapes · 9 classes · U-Net & transfer learning · FastAPI + Streamlit · Hugging Face Spaces

① Contexte, objectifs & principes (5 min)

- Problème métier et jeu de données
- Principe de la segmentation sémantique
- Mesures de performance

② Modèles & comparaisons (10 min)

- 4 architectures
- Protocole d'entraînement
- Résultats & analyse

③ Mise en production (5 min)

- Architecture API + application Web
- Déploiement Cloud + démonstration



Masque de segmentation

1 | Contexte & objectifs

Contexte métier

- Compréhension automatique de scènes urbaines : **véhicules autonomes**, **villes intelligentes**, cartographie.
- Jeu de données **Cityscapes** : images de rues (Allemagne), annotées au *pixel près*.
- Simplification à **8 classes** utiles : road, building, vegetation, sky, person, car, traffic_sign, bicycle + background (tout le reste).



Palette de type Cityscapes (1 couleur / classe)

Objectifs du projet

- ① Entraîner et **comparer plusieurs modèles** de segmentation.
- ② Sélectionner le meilleur via un suivi d'expériences (**MLflow**).
- ③ **Mettre en production** : API REST + application Web sur le Cloud.

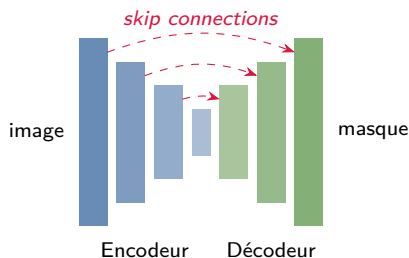
1 | Qu'est-ce que la segmentation sémantique ?

Classer chaque pixel de l'image dans une catégorie.

- Classification : 1 étiquette / *image*.
- Détection : 1 boîte / *objet*.
- **Segmentation** : 1 étiquette / *pixel* → sortie = image de même taille.

Principe : encodeur-décodeur

- **Encodeur** : extrait les caractéristiques, réduit la résolution (*quoi ?*).
- **Décodeur** : reconstruit la résolution d'origine (*où ?*).
- **Skip connections** (U-Net) : réinjectent les détails fins perdus au sous-échantillonnage.



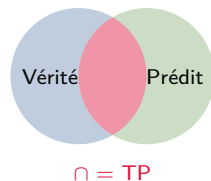
Architecture en « U » (U-Net)

Comment comparer objectivement les modèles ? — via la **matrice de confusion** par pixel.

IoU (Intersection over Union) — par classe :

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{TP}{TP + FP + FN}$$

- **mIoU** : moyenne des IoU sur les 9 classes → **métrique principale** (traite chaque classe à égalité, même les rares).
- **Pixel accuracy** : % de pixels bien classés → trompeur (dominé par les grandes classes : route, ciel).
- **IoU par classe** : diagnostic fin (ex. panneaux, vélos).



Pourquoi mIoU ? Robuste au fort déséquilibre des classes.

2 | Les 4 architectures comparées

Modèle	Encodeur	Idée clé
UNet	from scratch	Référence, aucun pré-entraînement
VGG16-UNet	VGG16 (ImageNet)	Transfer learning , encodeur classique
ResNet34-UNet	ResNet34 (ImageNet)	Connexions résiduelles
ResNet50-UNet	ResNet50 (ImageNet)	Encodeur profond → retenu

- Toutes partagent le même **décodeur de type U-Net** (skip connections).
- **Hypothèse** : un encodeur pré-entraîné sur ImageNet apporte des caractéristiques génériques (bords, textures) → meilleure segmentation.
- Code factorisé dans un package partagé cityvision/ (classes, palette, modèles) — *une seule source de vérité* pour l'entraînement et le service.

Données & prétraitement

- Cityscapes, redimensionné à **512×1024**.
- Normalisation **ImageNet** (cohérente avec les encodeurs pré-entraînés).
- Remappage des labelIds bruts → 9 classes.

Optimisation

- Optimiseur **Adam**, 20 epochs, **early stopping** (patience 5).
- **Learning rate différentiel** : encodeur $\times 0,1$, décodeur $\times 1$ (on ne « casse » pas les poids pré-entraînés).

Gestion du déséquilibre des classes

- Perte combinée **Cross-Entropy + Dice**, pondérée par classe (panneaux & vélos $\times 3$, personnes $\times 2,5$).
- **WeightedRandomSampler** : enrichit les batchs en images contenant des classes rares.

Suivi & reproductibilité

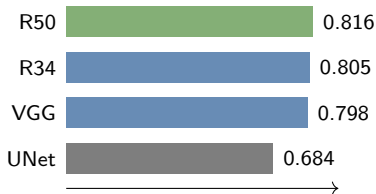
- **MLflow** : paramètres, métriques par epoch, IoU par classe, artefact du meilleur checkpoint.
- Comparaison *avec / sans augmentation* (flip horizontal + color jitter).

2 | Résultats : comparaison des modèles

Architecture	Encodeur	mIoU (validation)
ResNet50-UNet	ResNet50 (pré-entraîné)	0,816 ★
ResNet34-UNet	ResNet34 (pré-entraîné)	0,805
VGG16-UNet	VGG16 (pré-entraîné)	0,798
UNet	from scratch	0,684

Enseignement principal

Les encodeurs **pré-entraînés** surpassent nettement l'entraînement *from scratch* : **+0,13 de mIoU** (0,68 → 0,82).



Ce qui marche bien

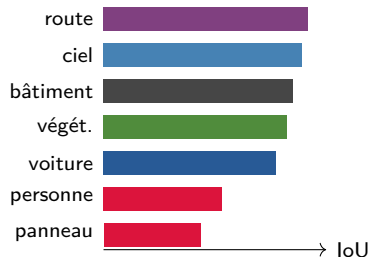
- Grandes classes structurantes : **route**, **bâtiment**, **ciel**, **végétation** → IoU élevée.

Ce qui reste difficile

- Petits objets fins et rares : **panneaux**, **vélos**, **personnes** → IoU plus faible.
- D'où les *contre-mesures* : pondération de la perte + échantillonnage équilibré + résolution 512×1024 .

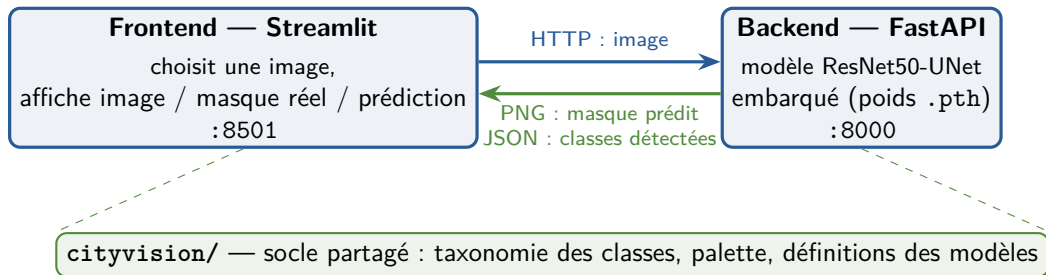
Effet de l'augmentation

- Flip + color jitter : gain modéré, surtout en robustesse (comparé dans les notebooks).



IoU par classe (tendance) : rare & petit = plus dur.

3 | Architecture : API + application Web



- **Découplage** client / service : le frontend ne fait *que* des appels HTTP.
- **Même code d'inférence qu'à l'entraînement** (prétraitement ImageNet identique) → pas de *training/serving skew*.

3 | L'API REST de prédiction

Méthode	Route	Rôle
GET	/health	état du service + modèle chargé
GET	/docs	documentation interactive Swagger
POST	/predict	image → masque JSON (dense/rle)
POST	/predict/image	image → masque PNG (color/overlay/raw)
POST	/predict/classes	image → classes détectées + % couverture

Choix de conception

- Modèle chargé **une fois au démarrage**.
- Inférence CPU en **threadpool** (boucle async libre).
- Encodage **RLE** : masque compact (grandes images).
- Garde-fous : limite d'upload, validation, logs JSON.

Exemple d'appel

```
curl -X POST \  
  "http://.../predict/image?\  
    format=overlay" \  
  -F "file=@scene.png" \  
  -o overlay.png
```

Conteneurisation

- Deux images **Docker** (backend / frontend), orchestrées en local par `docker compose`.
- PyTorch **CPU** + poids `.pth` **embarqués** dans l'image (via `git-lfs`).

Cloud choisi : **Hugging Face Spaces**

- **Deux Spaces** (SDK Docker) : un pour l'*API*, un pour l'*app*.
- L'app consomme l'API via la variable `CITYVISION_API`.
- Pourquoi ? *gratuit/simple, natif Docker, déploiement par git push*, idéal pour une démo publique.

Qualité & CI/CD

- **GitHub Actions** : `black` + `flake8` + `pytest` à chaque push/PR.
- Tests backend et frontend (tests/).

Démo en ligne

▶  Application

▶  API (Swagger)

(liens cliquables)

Scénario de démonstration

- 1 Ouvrir l'**application Streamlit**.
- 2 Sélectionner une image de rue.
- 3 L'app appelle `POST /predict/image` sur l'API.
- 4 Affichage côte à côte :
image réelle · **masque réel** · **masque prédit**.
- 5 Superposition (*overlay*) réglable + classes détectées.

image d'entrée



masque prédit

Lancer la démo

▶  Ouvrir l'application

▶  Ouvrir l'API (Swagger /docs)

i En repli : `curl` direct sur l'API.

Modélisation

- **Pourquoi U-Net ?** skip connections = récupération des détails fins, efficace sur peu de données.
- **Pourquoi le mIoU ?** insensible au déséquilibre, standard Cityscapes.
- **ResNet50 vs ResNet34 ?** +0,011 de mIoU pour un coût plus élevé — arbitrage qualité/latence.
- **Déséquilibre** traité par perte pondérée + Dice + sampler.

Production

- **HF Spaces vs AWS/GCP ?** simplicité & coût pour une démo ; migration possible (l'image Docker est portable).
- **Latence CPU ?** acceptable pour la démo ; GPU si passage à l'échelle.
- **Résolution 512×1024 ?** compromis mémoire / détails (petits objets).
- **Robustesse** : validation d'entrée, limite d'upload, logs structurés.

Pistes d'amélioration : modèles type DeepLab/Transformer, quantification, GPU, jeu de données élargi.

Merci — place à la discussion

CityVision

Segmentation urbaine → ResNet50-UNet (mIoU 0,816)
→ API FastAPI + app Streamlit → Hugging Face Spaces

 github.com/delnouty/cityvision-segmentation



App



API

Découplage frontend / backend

- Frontend **Streamlit léger** (sans PyTorch) ↔ backend **FastAPI** qui embarque le modèle.
- Déploiement & montée en charge **indépendants**; le frontend n'a aucune dépendance lourde.
- Contrat clair : le client ne fait que des appels HTTP.

Pourquoi **FastAPI**

- Inférence CPU en **threadpool** (boucle async non bloquée).
- Validation d'entrée, **OpenAPI** auto (/docs), garde-fous (limite d'upload).

Une API « style REST »

- **Sans état** : chaque requête est autonome, modèle chargé une fois au démarrage.
- Méthodes HTTP standard (GET /health, POST /predict...), codes de statut (413, 503), représentations **JSON & PNG**.
- **Interopérable** : tout client (curl, Streamlit, navigateur).

Nuance honnête : REST *pragmatique*

- Points d'entrée **orientés action** (/predict, /predict/image) plutôt qu'orientés ressource (/predictions/{id}).
- Pas de *HATEOAS*, pas de ressource adressable persistante — inutile pour une inférence sans état.
- Compromis assumé : **simplicité** > pureté REST pour un service d'inférence.

Critères de sélection

- Démo **publique**, simple et à **faible coût**.
- Déploiement **reproductible** (Docker) et rapide (git push).
- Écosystème orienté **ML / IA**.

Pourquoi Hugging Face Spaces

- **SDK Docker natif** : la même image qu'en local, pas de refonte.
- Déploiement par git push ; URL publique + HTTPS fournis.
- **Deux Spaces** : API (FastAPI) + app (Streamlit), reliés par CITYVISION_API.
- API en **always-on** (payant) pour éviter les *cold starts* ; app sur l'offre gratuite.

Alternatives considérées

- **AWS / GCP / Azure** : plus puissants (GPU, autoscaling) mais configuration & coût plus lourds pour une simple démo.
- **Render / Railway** : proches en simplicité, mais moins orientés ML et sans écosystème de modèles.

Compromis assumés

- **CPU only** : latence acceptable pour la démo (GPU si passage à l'échelle).
- Dépendance à HF, mais l'**image Docker est portable** → migration possible sans réécriture.